

High Performance Data Analytics and Visualisation

Assignment: Housing Data Visual - Report

Matteo Arrigo

October 26, 2025

1 Introduction

This report describes the visual design and implementation of an interactive multi-view visualization system for housing market data. The system combines two coordinated views: a 2D Scatterplot (40% width) and Parallel Coordinates Plot (PCP) (60% width) enabling multivariate analysis across all 13 dataset attributes. The focus of this implementation was the PCP component and the coordinated brushing interaction between views.

2 Data Characteristics

The housing dataset contains 545 records with 13 attributes of varying types: continuous numeric (**price**, **area**), discrete numeric (**bedrooms**, **bathrooms**, **stories**, **parking**), and categorical (**mainroad**, **guestroom**, **basement**, **hotwaterheating**, **airconditioning**, **prefarea**, **furnishingstatus**). This heterogeneous structure requires different visual encodings and scale types to properly represent each attribute's semantic meaning.

3 Visual Encoding Design

3.1 Scatterplot (Brief Overview)

The scatterplot uses standard 2D position encoding to map **area** to the x-axis and **price** to the y-axis, leveraging position as the strongest perceptual channel for quantitative comparison, while colored strokes distinguish selected samples. This view provides an intuitive entry point for exploring the primary price-area correlation and detecting outliers.

3.2 Parallel Coordinates Plot (PCP) Design

The PCP implementation was developed from scratch to handle the high-dimensional nature of the dataset.

3.2.1 Multi-Type Scale System

A critical challenge was representing three distinct data types within a single visualization framework. The solution implements dimension categorization that analyzes each attribute to determine whether it should use: (1) **Linear scales** (`d3.scaleLinear()`) for continuous data like price and area, preserving magnitude relationships; (2) **Point scales** (`d3.scalePoint()`) for discrete numeric attributes (e.g., bedrooms) showing distinct values; or (3) **Point scales** for categorical attributes with string values, enabling label display.

3.2.2 Interactive Dimension Control

To manage 13 dimensions without visual clutter, the PCP provides three complementary interaction mechanisms:

- **Dynamic axis count control** via a number input (range 2-13, default 6) positioned below the visualization. It allows users to add or remove axes on demand: when axes are added, dimensions are selected sequentially from the full attribute list; when removed, the rightmost axes are eliminated.
- **Dropdown menus** on each axis enable selection from all 13 available dimensions for flexible dimension substitution. The implementation embeds HTML dropdowns within SVG using `foreignObject` elements
- **Drag-and-drop reordering** via drag handles above each dropdown, implemented using D3's drag behavior. Indeed, reordering is crucial because PCP patterns depend heavily on axis adjacency.

3.2.3 Visual Encoding Strategy

Like the classical PCP seen during the lectures, position encoding is used for both dimensions (X-axis, for the different dimensions to show) and values (Y-axis, quantitative/categorical per dimension). Line color, opacity and stroke width indicates selection state. In particular, low opacity for unselected lines reduces clutter with 545 polylines.

State	Color	Opacity	Stroke width
Unselected	#B0BEC5 (grey)	0.2	1.2px
Selected	#C62828 (red)	0.8	1.8px
Highlighted (hover)	#F9A825 (yellow)	1	3px

The 60% width allocation provides adequate space for at least 6 axes while maintaining readability.

3.3 Coordinated Brushing Interaction

The key interaction is the bidirectional brushing between views. A 2D brush (`d3.brush()`) on the scatterplot enables rectangular region selection, with selected points highlighted in both views. Each PCP axis features an independent 1D vertical brush (`d3.brushY()`), with behavior varying by dimension type: linear dimensions define continuous ranges using `scale.invert()`, while categorical/point dimensions capture discrete values within the brush extent as a value set. Multiple brushes can be active simultaneously, with filtering applied conjunctively (AND filters). When a brush is activated in either view, the other view's brush is automatically cleared to maintain clarity.

Also, both views support single-item selection via click. In the scatterplot, clicking a point toggles its membership in the current selection (it is added or removed) without replacing other selected items. In the PCP, where line overlap is common due to the high number of discrete values, clicking a polyline defines a new filter: the values at the line's terminal axes (left and right dimensions of the selected segment) are used to select all matching records, and this action replaces any prior selection.

4 Advantages and Limitations

4.1 Parallel Coordinates Strengths

The PCP effectively can display all 13 attributes simultaneously, far exceeding scatterplot capabilities. Parallel lines indicate correlation while crossing patterns reveal inverse relationships. The dropdown and drag interface enables easy exploration of all possible axis arrangements, and multi-axis brushing enables complex subset definition.

4.2 Parallel Coordinates Limitations

Despite opacity adjustments, rendering 545 polylines still generates substantial visual clutter: numerous discrete values produce heavy line overlap that can obscure multivariate patterns. Readability is highly sensitive to axis ordering and to the fact that each dimension uses its own y-scale, which makes PCPs harder to interpret than simple scatterplots. Mitigations implemented include interactive brushing to reduce noise, drag-and-drop axis reordering to expose adjacency-dependent patterns, and per-axis dropdowns to substitute or aggregate dimensions for targeted exploration.

4.3 Alternative Approaches Considered

Several alternative visualizations were considered but not adopted due to scalability or fidelity trade-offs.

For example, a pairwise scatterplot exposes bivariate relationships but, with 13 attributes, would produce 78 cells and severe overplotting for 545 records; interactive enhancements mitigate this somewhat but remain cumbersome for high-dimensional exploration. Instead, tree- or aggregation-based layouts reveal group structure but abstract away the per-record detail required for axis-based brushing, so they were deprioritized.

Conclusion: The PCP with coordinated brushing best balances per-record multivariate comparison and interactive exploration for this dataset.

5 Implementation Patterns and Design Choices

5.1 React Component Architecture and State Management

The application follows a component-based architecture with four main React components: `App` (parent), `ScatterplotContainer`, `ParallelCoordinatesContainer` and `AxisCountControl`. State management follows React's unidirectional data flow principle with `useState` hooks in the parent component. The `App` component maintains global state including housing data, selected items and number of visible axes. State flows downward via props to child components, while state updates flow upward via callback functions.

Inter-component communication is achieved through state lifting and callback registration patterns. A complete example of this is the coordinated brushing interaction.

5.1.1 Coordinated brushing

Cross-view brush coordination is achieved through React hooks (`useRef`, and `useEffect`). Each visualization container exposes its brush-clearing function by assigning it to a ref provided by the parent component during mount. The parent component maintains refs to both views' clear functions (`scatterplotClearBrushRef`, `parallelCoordinatesClearBrushRef`).

Controller method objects provide an interface between D3 events and React state. When a brush activates in one view, the D3 event handler calls a controller method that invokes the other view's clear function via its ref. For selection updates, D3 events trigger controller methods that update the parent's `selectedItems` state via `setSelectedItems()`. This state change propagates back to both containers as props, where `useEffect` hooks detect the update and call D3 highlighting methods. This maintains React's unidirectional data flow while enabling coordinated interaction through a simple ref-based communication pattern.

5.2 React and D3 Separation of Responsibilities

The implementation maintains strict separation between React (state management) and D3 (visualization rendering). React containers handle lifecycle management via `useEffect` hooks for component mounting / unmounting and D3 instance creation/cleanup. The `useRef` hook persists D3 instances across renders. Controller method objects bridge D3 callbacks to React state updates without D3 directly modifying state.

D3 classes own all SVG creation, DOM manipulation, scales, and axes. Rendering logic uses data binding with `.data()`, `.join()`, and `.enter()` patterns for efficient enter/update/exit handling. D3 event listeners (drag, brush, click) callback to React via provided controller methods. This architecture ensures maintainable code: React owns component lifecycle and application state, D3 owns visual rendering and interaction logic, and they communicate through a clean callback interface.

5.3 Other Implementation Decisions

Brush ranges are stored with composite keys (`dimension_axisIndex`) to support dynamic axis reordering. All D3 callbacks route through controller method objects, maintaining unidirectional data flow (React → D3 → React callbacks). Responsive sizing uses container dimensions (`offsetWidth/Height`) rather than hard-coded values. CSV parsing includes empty row filtering to prevent null value issues discovered during development.

6 Conclusion

This visualization system balances exploratory power with usability through type-aware scales, flexible dimension control, and multi-axis brushing in the parallel coordinates view. The coordinated interaction provides bidirectional filtering while maintaining visual context. The architectural separation between React and D3 ensures maintainable code following established patterns, enabling independent evolution of visualization logic without affecting state management. The 60/40 width allocation reflects the relative importance of multivariate analysis versus bivariate correlation for this dataset.